SERVICE AUTO

Proiectarea aplicatiilor web · Proiect individual (examen)

Platforma web pentru administrarea si programarea lucrarilor intr-un service auto

Camp	Detalii
Student	Victor Strimbeanu
Grupa	CR4H
Aplicatie	Service Auto
Data	5 iunie 2026
Tehnologii	HTML, CSS, JavaScript, Bootstrap 5, ASP.NET Core MVC, EF Core, SQLite

Cuprins

1. Rolul, obiectivele si cerintele aplicatiei
2. Modelarea aplicatiei prin diagrame UML
3. Arhitectura sistemului si baza de date
4. Functionalitati implementate si capturi de ecran
5. Testare, problema identificata si solutie
6. Plan de promovare
7. Utilizabilitate si accesibilitate
8. Securitate
9. Limitari, puncte slabe si extinderi
10. Autoevaluare
Anexa. Structura repository-ului

1. Descrieti rolul si obiectivele aplicatiei. Specificati in detaliu cerintele aplicatiei.

Rolul aplicatiei

Aplicatia Service Auto a fost construita pentru a muta o parte din activitatea unui service auto intr-un mediu web usor de folosit. In loc ca toate programarile si datele despre clienti sa fie tinute telefonic sau in fisiere separate, aplicatia centralizeaza informatiile intr-o baza de date si ofera un traseu clar: clientul se informeaza, isi creeaza cont, isi adauga masina si poate trimite o cerere de programare online.

Sistemul are doua zone care se completeaza intre ele. Prima zona este site-ul public, realizat cu HTML, CSS, JavaScript si Bootstrap, unde clientul vede serviciile, preturile, datele de contact si formularele de cont/programare. A doua zona este aplicatia ASP.NET Core MVC, folosita ca panou administrativ pentru lucrul cu entitatile din baza de date: clienti, vehicule, angajati, servicii si programari.

Obiectiv	Cum se regaseste in aplicatie
Informarea clientului	Pagini de prezentare pentru servicii, preturi, program si contact.
Programare online	Formular disponibil dupa autentificare, cu alegerea vehiculului, datei, intervalului orar si serviciului dorit.
Autonomie pentru client	Cont personal, profil, lista de masini si posibilitatea de anulare a programarilor.
Administrare interna	CRUD complet in panoul MVC pentru resursele principale ale service-ului.
Persistenta si trasabilitate	Date salvate in SQLite prin Entity Framework Core, cu migrari si seed initial.
Integrare frontend-backend	Site-ul static comunica prin API REST cu backend-ul ASP.NET Core.

Cerinte functionale

ID	Cerinta	Implementare
F1	Pagina principala cu prezentarea service-ului si buton de programare.	frontend/index.html
F2	Catalog cu serviciile auto oferite.	frontend/services.html
F3	Lista de preturi orientative pentru lucrari uzuale.	frontend/prices.html
F4	Pagini informative Despre si Contact.	frontend/about.html, frontend/contact.html
F5	Inregistrare client cu nume, email, telefon si parola.	POST /api/auth/register
F6	Autentificare client si memorarea sesiunii in browser.	POST /api/auth/login, localStorage
F7	Profil client cu date personale, vehicule si programari.	GET/POST/PUT/DELETE

ID	Cerinta	Implementare
		/api/profile/*
F8	Creare programare online numai pentru client autentificat.	POST /api/appointments
F9	Navigare responsive si meniu adaptat dupa autentificare.	Bootstrap 5, style.css, main.js
A1-A6	CRUD pentru servicii, clienti, vehicule, angajati, programari si legatura programare-serviciu.	Controllers MVC + Views Razor
A7-A8	Migrari automate si seed pentru catalogul de servicii.	Program.cs, ServiceTypesSeeder

Cerinte non-functionale

- Arhitectura pe straturi: Controller → Service → Repository → DbContext.
- Validare prin Data Annotations si verificari ModelState in controllere.
- Parole stocate hash-uit cu IPasswordHasher<Customer>, nu in clar.
- CORS configurat pentru integrarea frontend-ului static in mediul de dezvoltare.
- SQLite cu mod WAL si busy_timeout pentru un comportament mai stabil la acces concurent.

2. Modelati aplicatia folosind diagrame UML

Modelarea urmareste fluxul real al aplicatiei: clientul intra pe site, isi creeaza cont, isi adauga masina si trimite programarea, iar administratorul lucreaza ulterior in panoul MVC. Diagramele sunt adaptate la entitatile si endpoint-urile proiectului, nu sunt doar exemple generale.

2.1 Diagrama de cazuri de utilizare

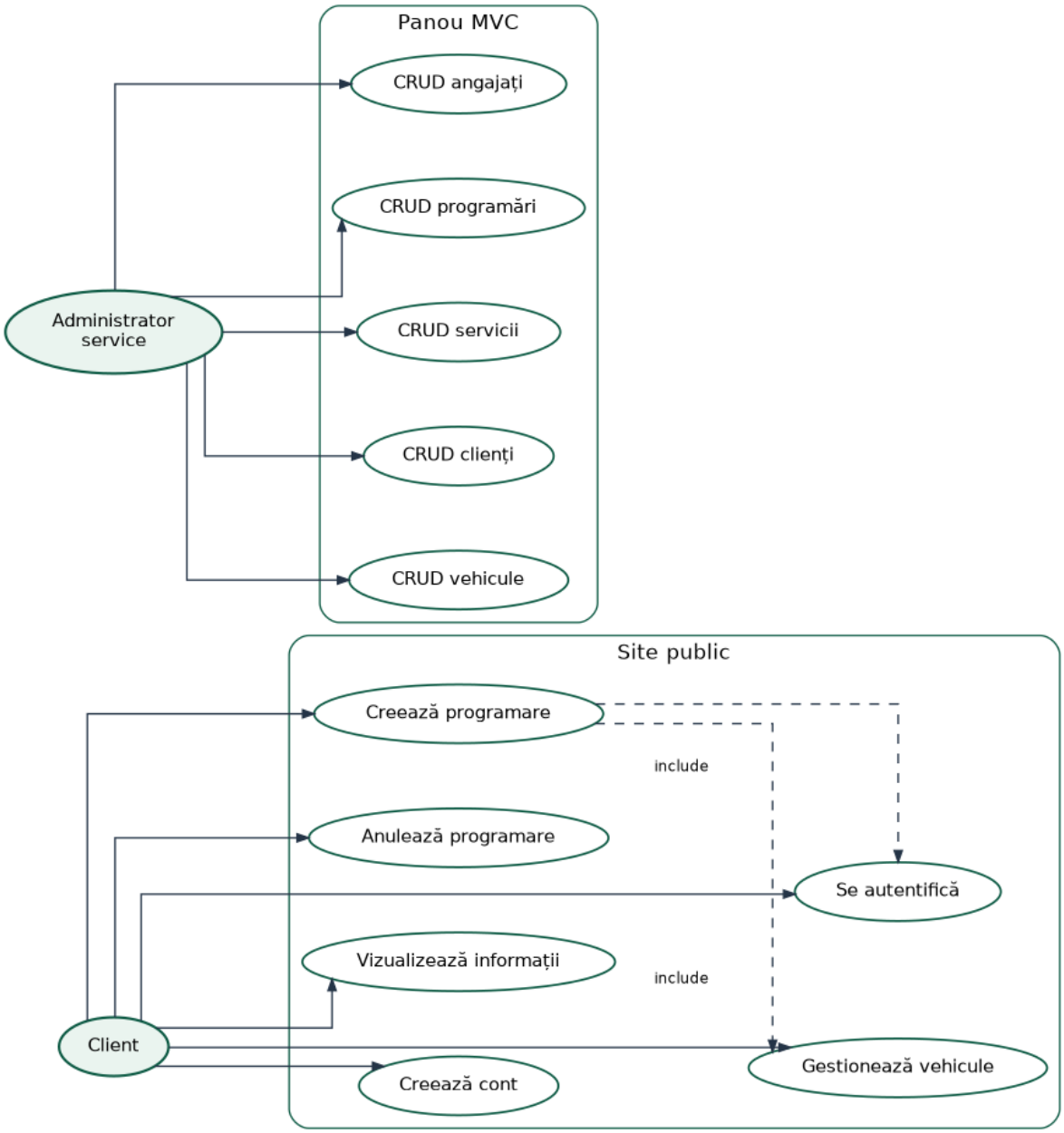


Figura 1. Cazuri de utilizare pentru client si administrator.

Pentru programarea online am considerat obligatorii autentificarea si existenta cel putin a unui vehicul in profil. Astfel, cazul „Creeaza programare” include implicit verificarea contului si a masinii selectate.

2.2 Diagrama de activitati - programare online

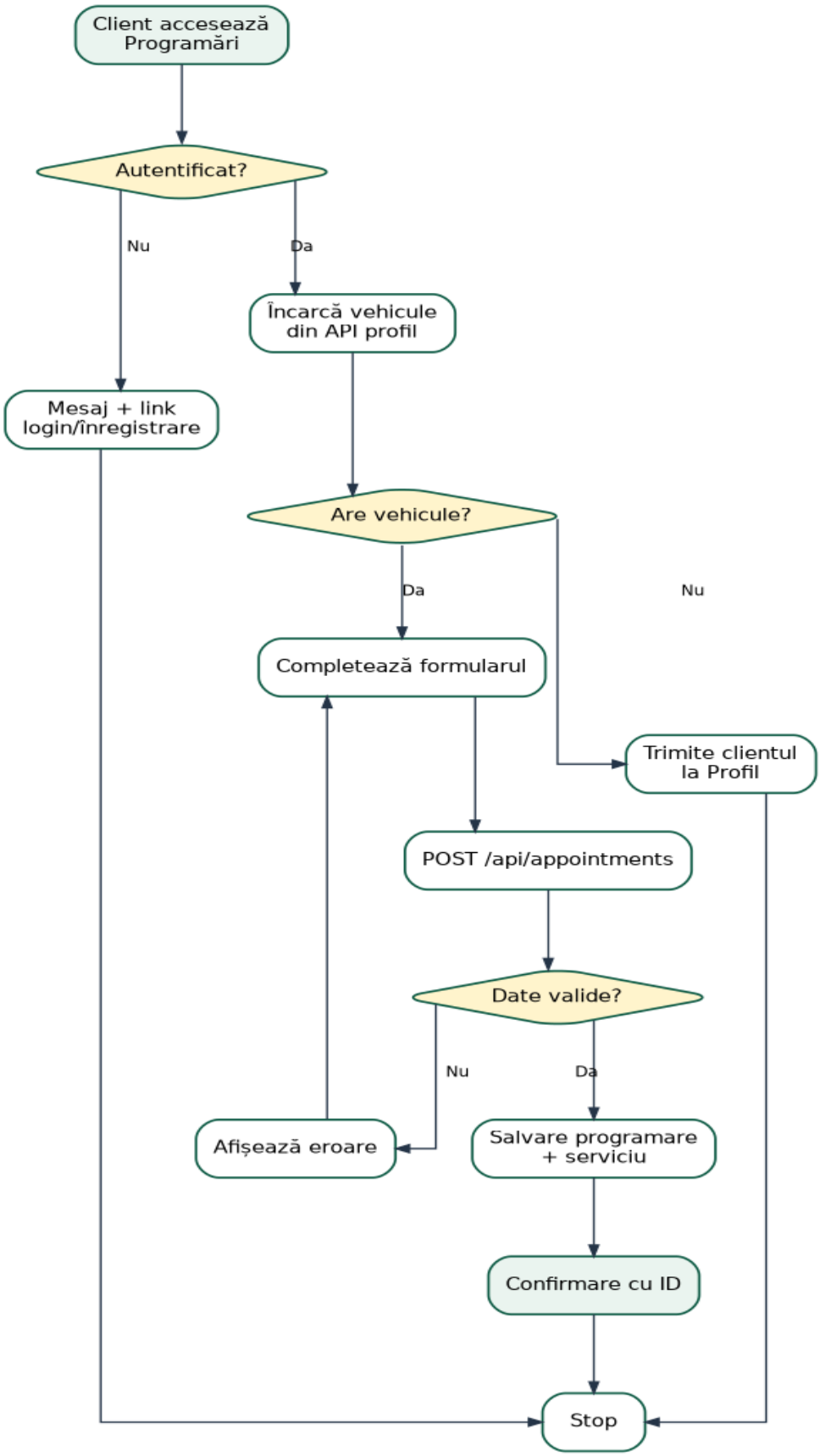


Figura 2. Pasii parcursi de client in formularul de programare.

2.3 Diagrama de clase

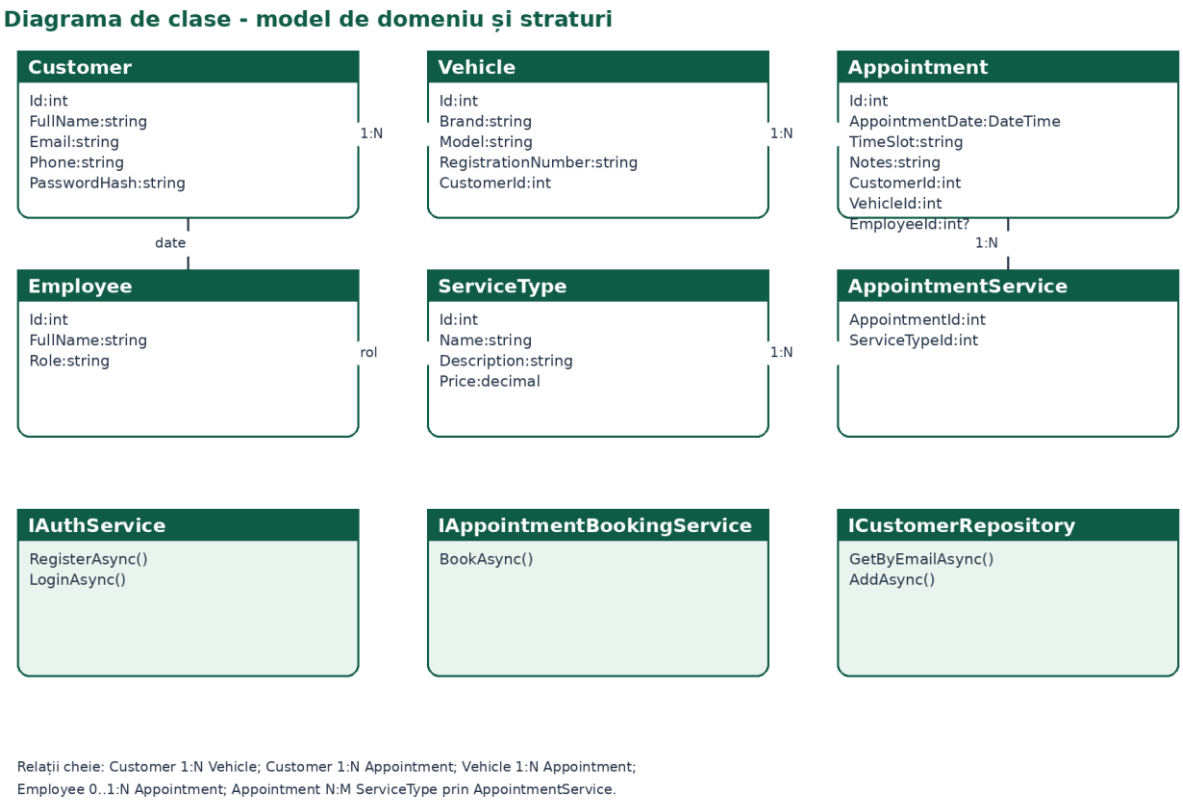


Figura 3. Modelul de domeniu si principalele servicii/repository-uri.

2.4 Diagrama de stare - programare

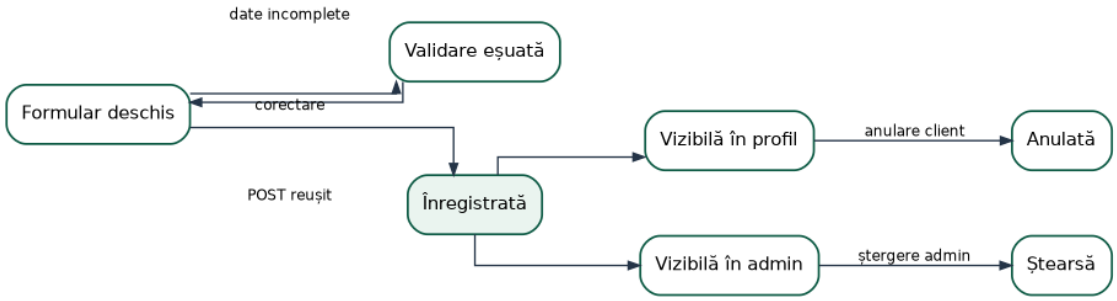


Figura 4. Starile unei programari de la formular pana la anulare/stergere.

2.5 Model de structura hipertext

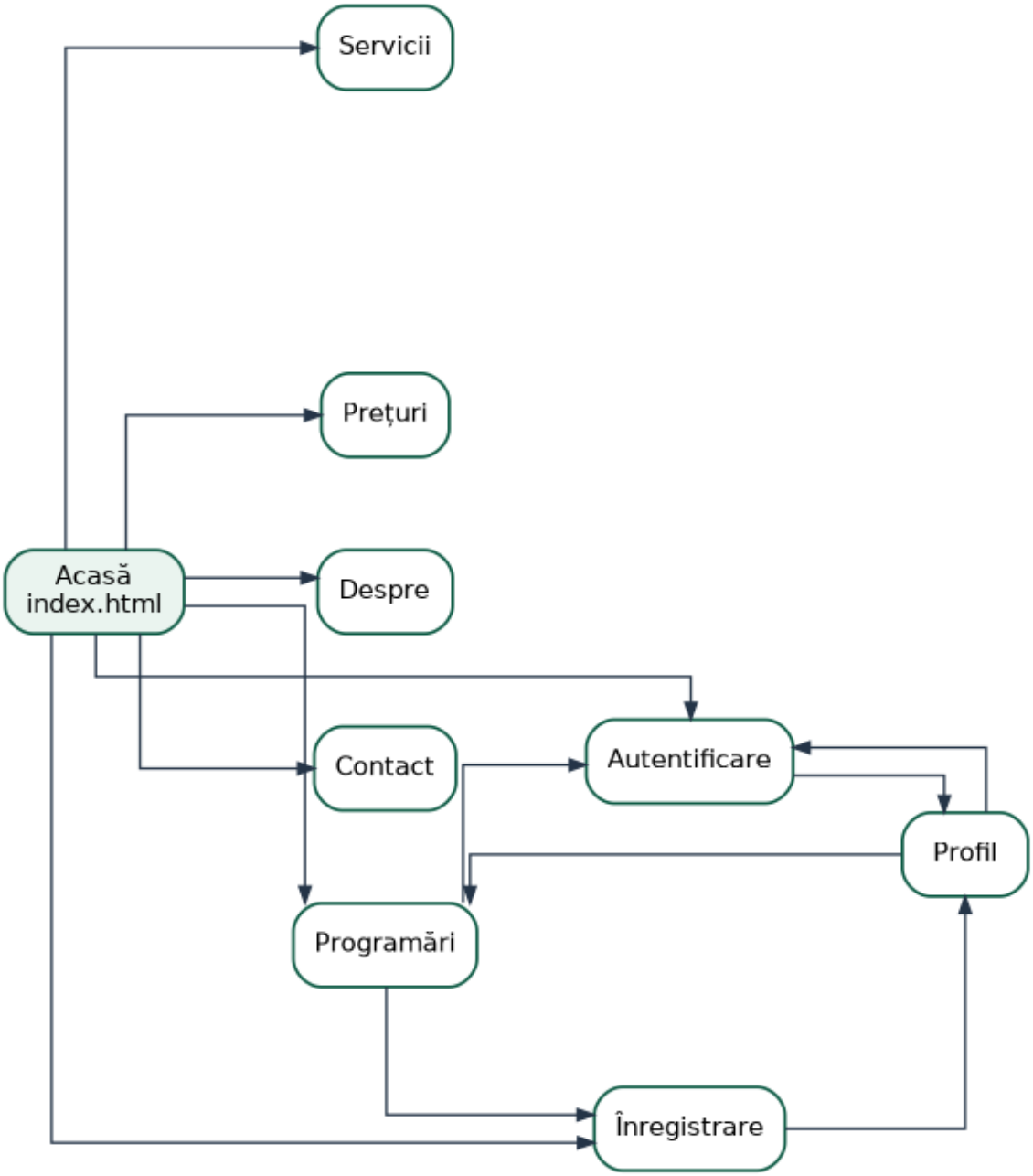


Figura 5. Legaturile principale dintre paginile site-ului public.

2.6 Model de acces

Resursa / zona	Vizitator neautentificat	Client autentificat	Administrator MVC
Pagini informative	Citire	Citire	-
Inregistrare / Login	Da	-	-

Resursa / zona	Vizitator neautentificat	Client autentificat	Administrator MVC
Profil si vehicule	Redirect la login	CRUD propriu	-
Programari publice	Formular dezactivat	Creare programare	-
Panou MVC	Acces tehnic nesecurizat*	Acces tehnic nesecurizat*	CRUD complet

2.7 Pagina de prezentare - wireframe conceptual

Zona	Elemente afisate
Header	Logo Service Auto, meniu: Acasa, Servicii, Programari, Preturi, Despre, Contact, Login/Profil.
Hero	Titlu de bun venit, descriere scurta, buton „Programeaza-te acum”.
Avantaje	Carduri pentru experienta, preturi corecte si garantie.
Servicii populare	Carduri cu servicii frecvente si butoane de detalii/programare.
Footer	Date de contact, program si linkuri utile.

2.8 Diagrama de interactiune - programare online

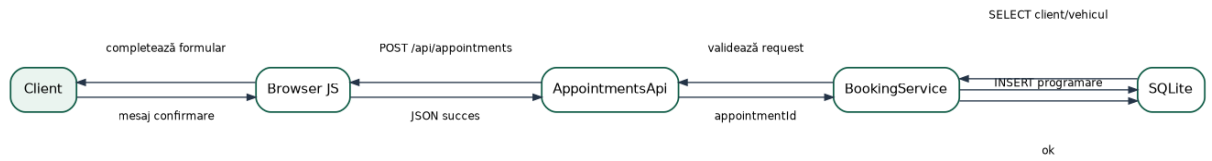


Figura 6. Interactiunea dintre browser, API, service si baza de date.

2.9 Diagrama de secventa - autentificare client

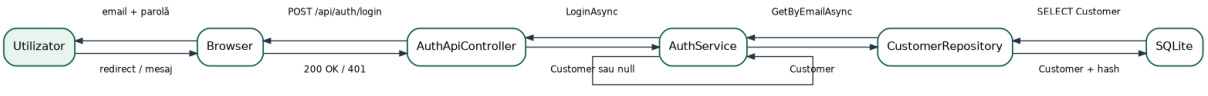


Figura 7. Secventa tehnica pentru login.

3. Descrieti arhitectura sistemului si structura bazei de date (inclusiv diagrame).

3.1 Arhitectura sistemului

Aplicatia este organizata pe straturi, pentru ca logica de prezentare, logica de business si accesul la date sa nu fie amestecate. Site-ul public consuma endpoint-uri JSON, in timp ce zona MVC foloseste controllere si view-uri Razor pentru administrare. Ambele zone ajung insa in acelasi strat de servicii si in aceeasi baza de date.

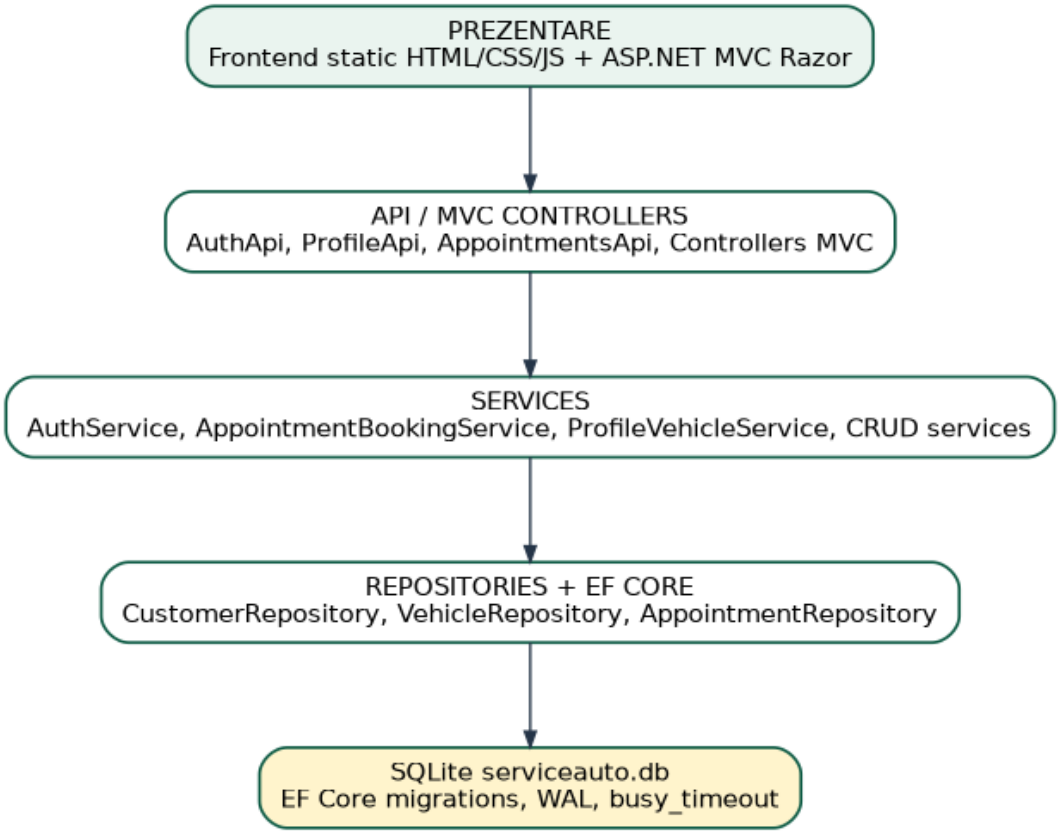


Figura 8. Arhitectura pe straturi a proiectului Service Auto.

Strat	Tehnologie / rol
Frontend public	HTML5, CSS3, JavaScript ES modules, Bootstrap 5
Backend	.NET 8, ASP.NET Core MVC + Web API
ORM	Entity Framework Core 8, abordare code-first
Baza de date	SQLite, fisier serviceauto.db
Validare	Data Annotations si ModelState
Parole	Microsoft.AspNetCore.Identity.PasswordHasher

Initializare la pornire

- Inregistrarea prin Dependency Injection a repository-urilor si serviciilor.
- Aplicarea migrarilor prin Database.Migrate().
- Activarea setarilor SQLite PRAGMA journal_mode=WAL si busy_timeout=60000.
- Upgrade incremental pentru coloane introduse ulterior, de exemplu TimeSlot si PasswordHash.
- Populareа catalogului de servicii cu ServiceTypesSeeder.

3.2 Structura bazei de date

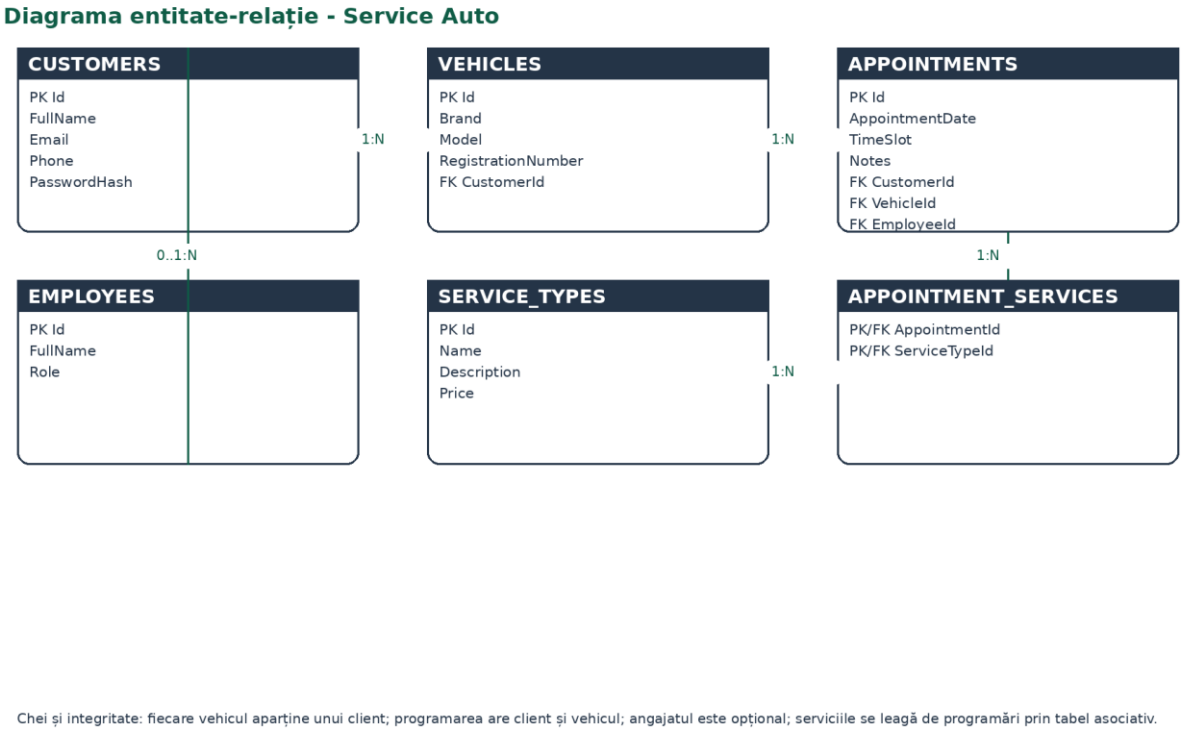


Figura 9. Diagrama entitate-relatie pentru baza de date.

Tabel	Rol in aplicatie	Relatii principale
Customers	Clientii service-ului si conturile acestora.	1:N cu Vehicles si Appointments
Vehicles	Masinile asociate clientilor.	N:1 cu Customers, 1:N cu Appointments
Employees	Angajatii care pot fi alocati lucrarilor.	1:N optional cu Appointments
ServiceTypes	Catalogul serviciilor si preturilor.	N:M cu Appointments prin AppointmentServices
Appointments	Programarile propriu-zise.	FK catre Customer,

Tabel	Rol in aplicatie	Relatii principale
		Vehicle, Employee
AppointmentServices	Tabel asociativ programare-serviciu.	Cheie compusa AppointmentId + ServiceTypeId

Reguli de integritate

- Fiecare vehicul trebuie sa apartina unui client existent.
- O programare trebuie sa aiba client si vehicul, dar angajatul poate fi completat ulterior.
- Legatura AppointmentService foloseste cheie primara compusa pentru a evita dublurile.
- Coloana TimeSlot separa intervalul orar de data programarii, ceea ce simplifica afisarea in interfata.

4. Descrieti functionalitatile pe care le-ati implementat si ilustrati-le cu capturi de ecran (screenshots).

Functionalitatile au fost grupate in doua zone: site-ul public pentru clienti si panoul MVC pentru administrare. Capturile de ecran de mai jos arata componentele implementate in proiect.

4.1 Site public - pagini informationale

Pagina principala foloseste un hero section cu mesaj clar si un call-to-action catre programari. Sub acesta apar avantajele service-ului si cateva servicii populare.

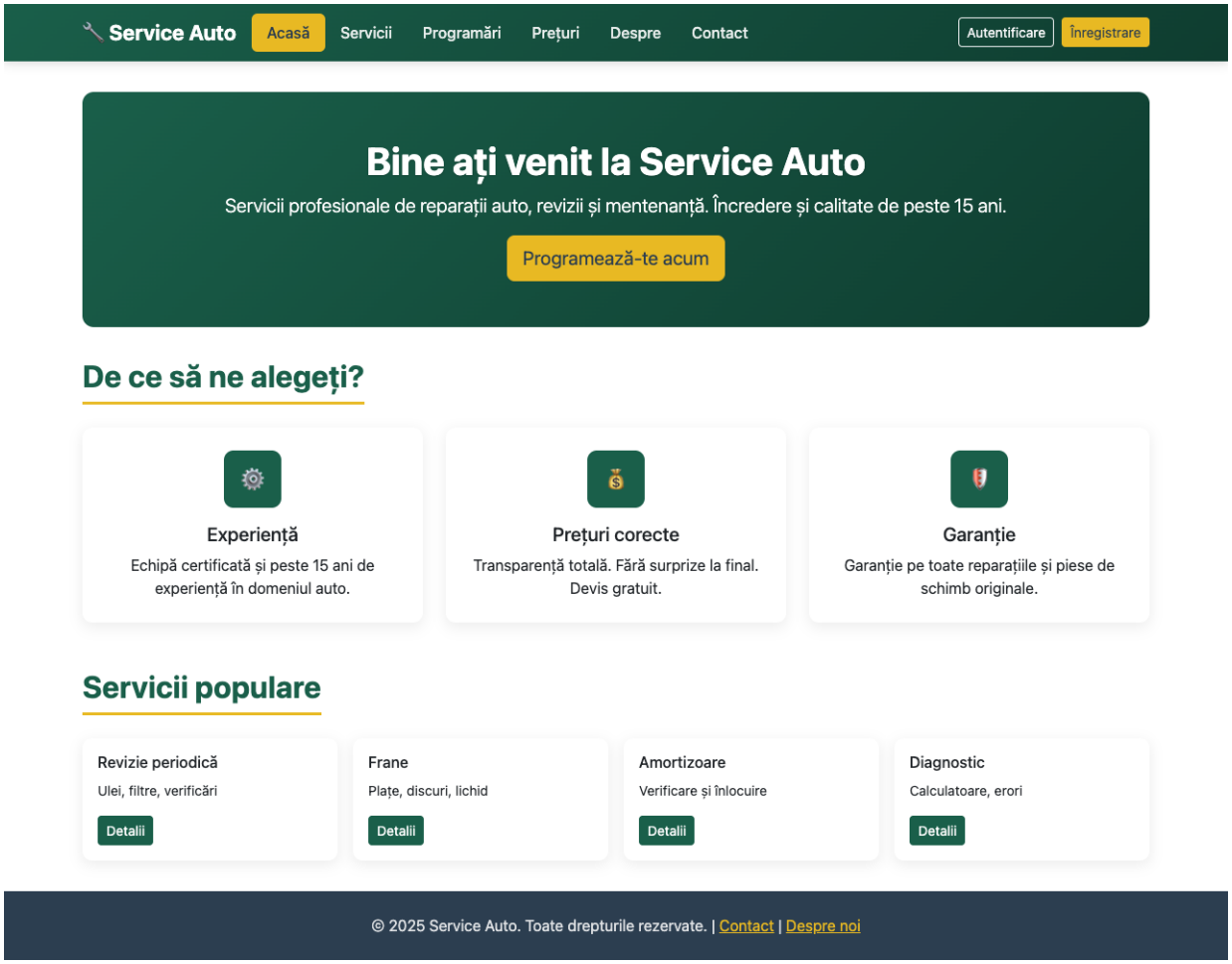


Figura 10. Pagina principala a site-ului Service Auto.

Pagina de servicii prezinta catalogul intr-un format de carduri, cu descrieri scurte si butoane de programare.

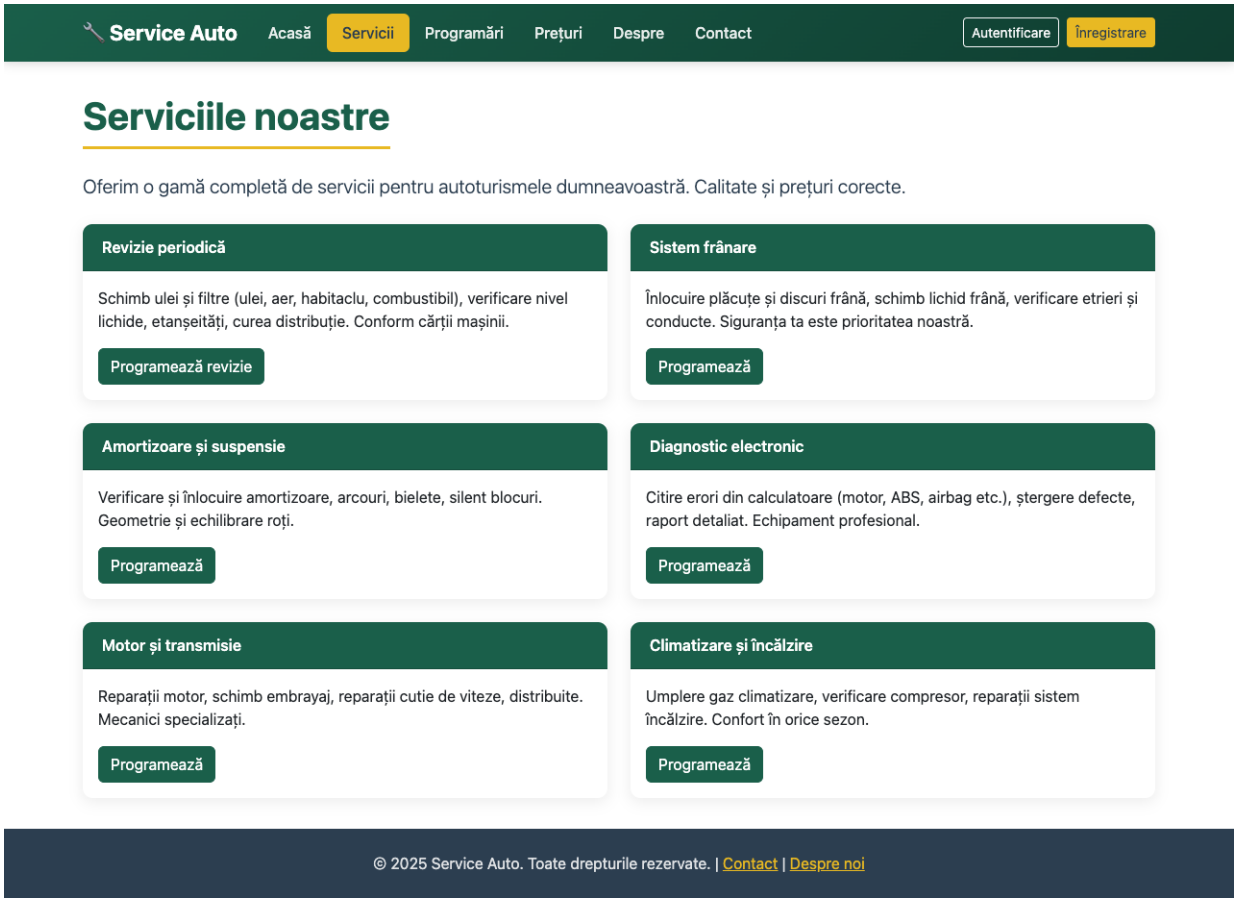


Figura 11. Catalogul de servicii disponibil pe site.

4.2 Autentificare si inregistrare

Clientul isi poate crea cont cu date de baza. Pe server parola este hash-uita, iar la autentificare browserul pastreaza datele sesiunii in localStorage pentru a adapta meniul si formularele.

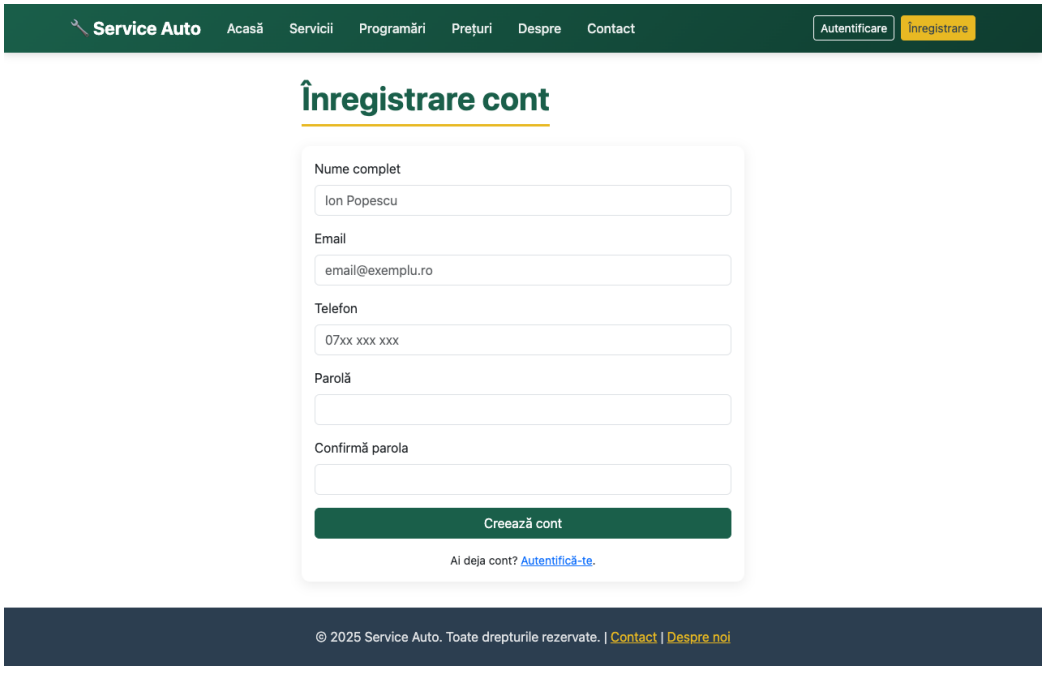


Figura 12. Formular de inregistrare client.

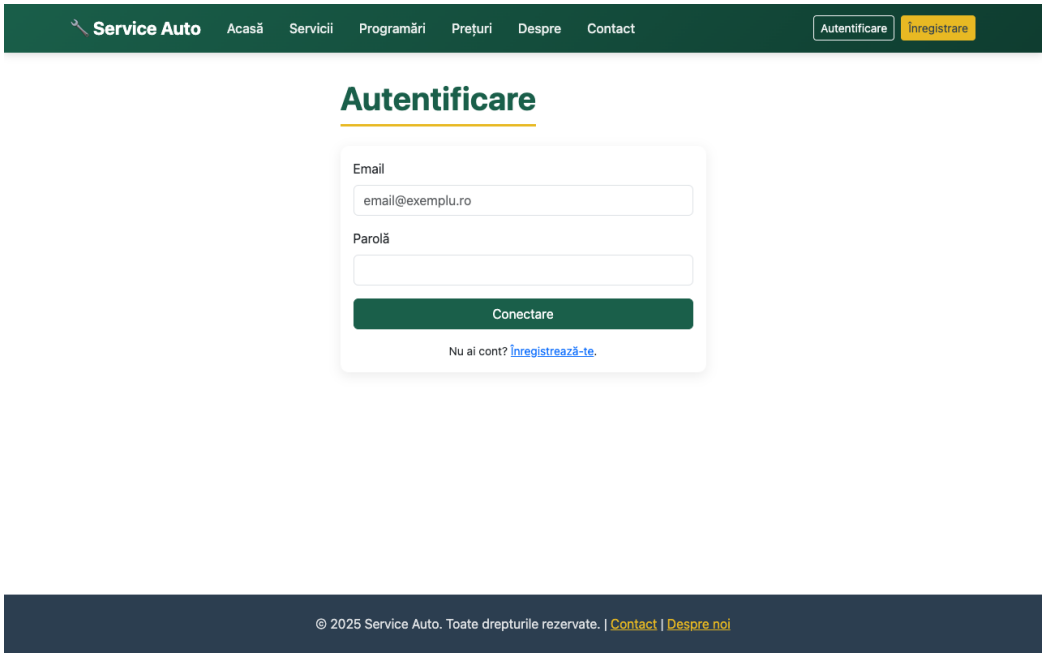


Figura 13. Formular de autentificare.

4.3 Profil client

Dupa login, clientul poate vedea datele personale, poate adauga sau modifica masini si poate urmari programarile existente. Aceasta pagina este importanta deoarece formularul de programare are nevoie de cel putin un vehicul salvat.

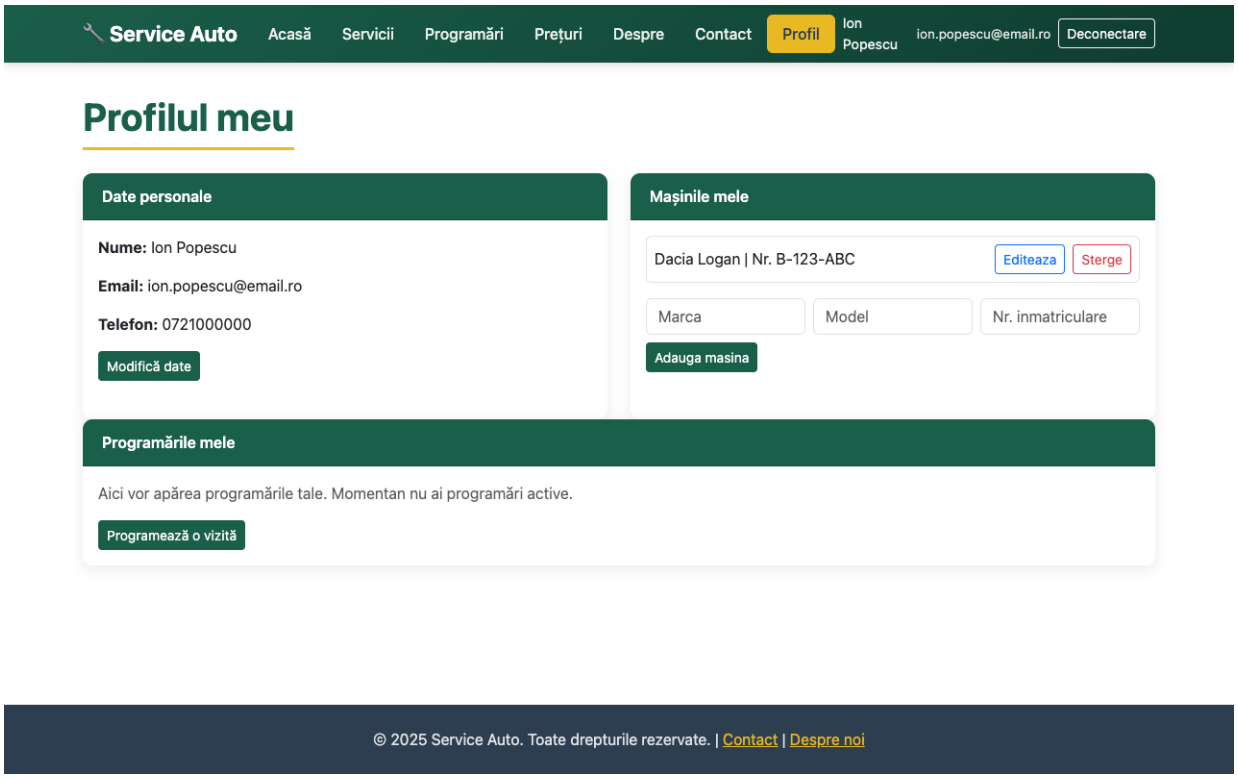


Figura 14. Profil client cu date personale, vehicule si programari.

4.4 Programare online

Formularul de programare preia datele clientului autentificat, permite alegerea datei, intervalului orar, serviciului si vehiculului. La trimitere, backend-ul valideaza masina si creeaza programarea impreuna cu serviciul ales.

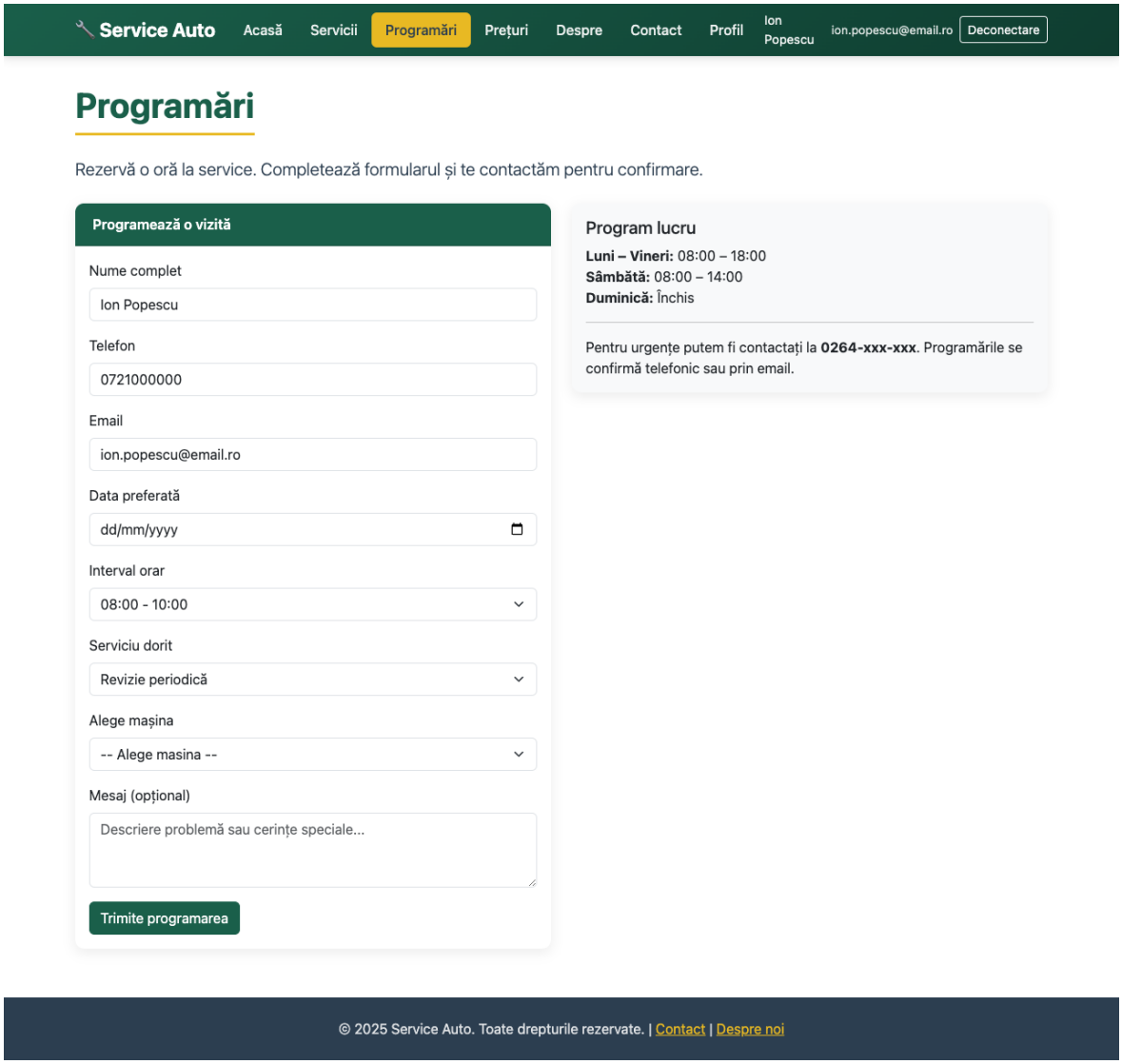


Figura 15. Formular de programare pentru utilizator autentificat.

4.5 Panou administrativ MVC

Panoul MVC este partea de administrare interna. El permite gestionarea directa a datelor din baza de date, separat de site-ul public.

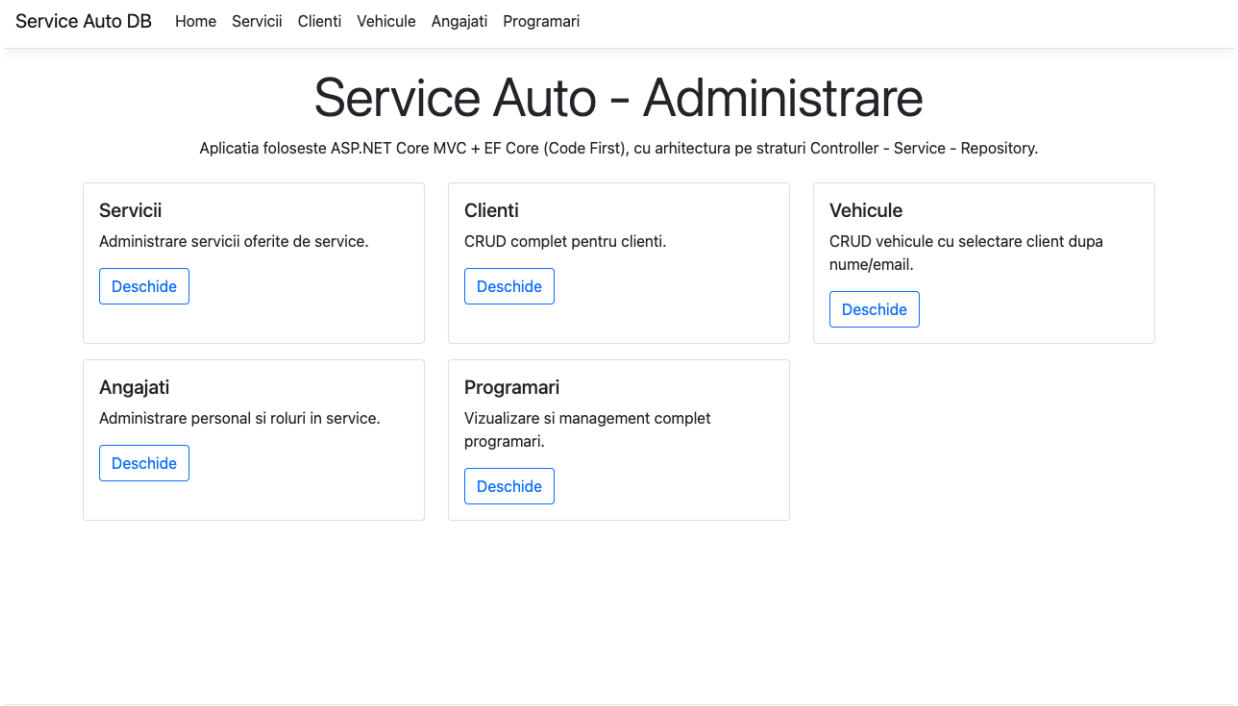


Figura 16. Dashboard-ul MVC al aplicatiei Service Auto DB.

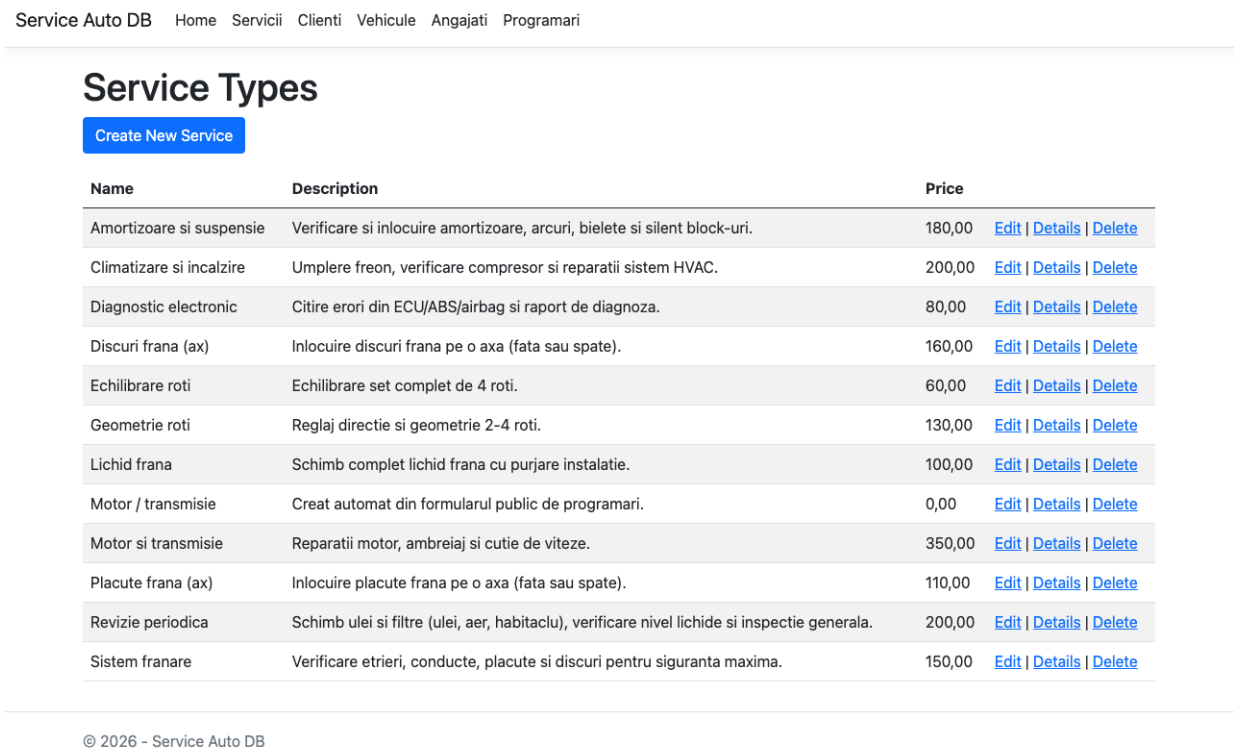


Figura 17. CRUD pentru tipurile de servicii.

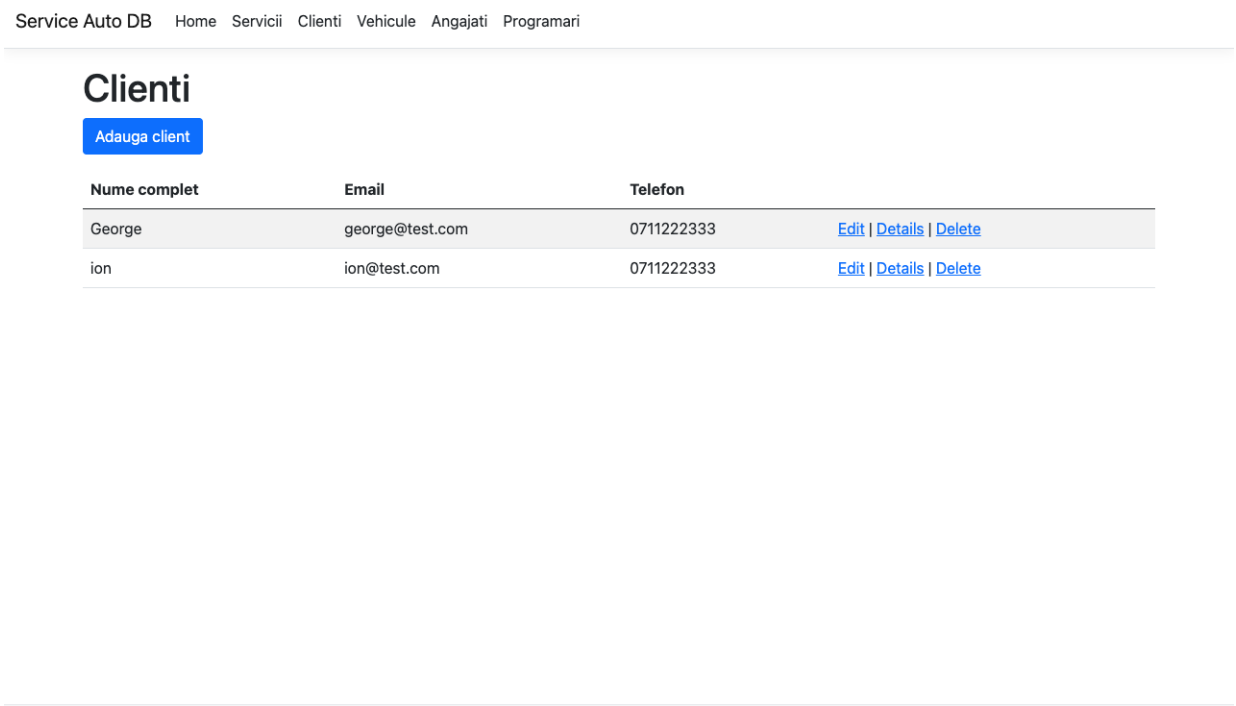


Figura 18. Listarea clientilor in panoul MVC.

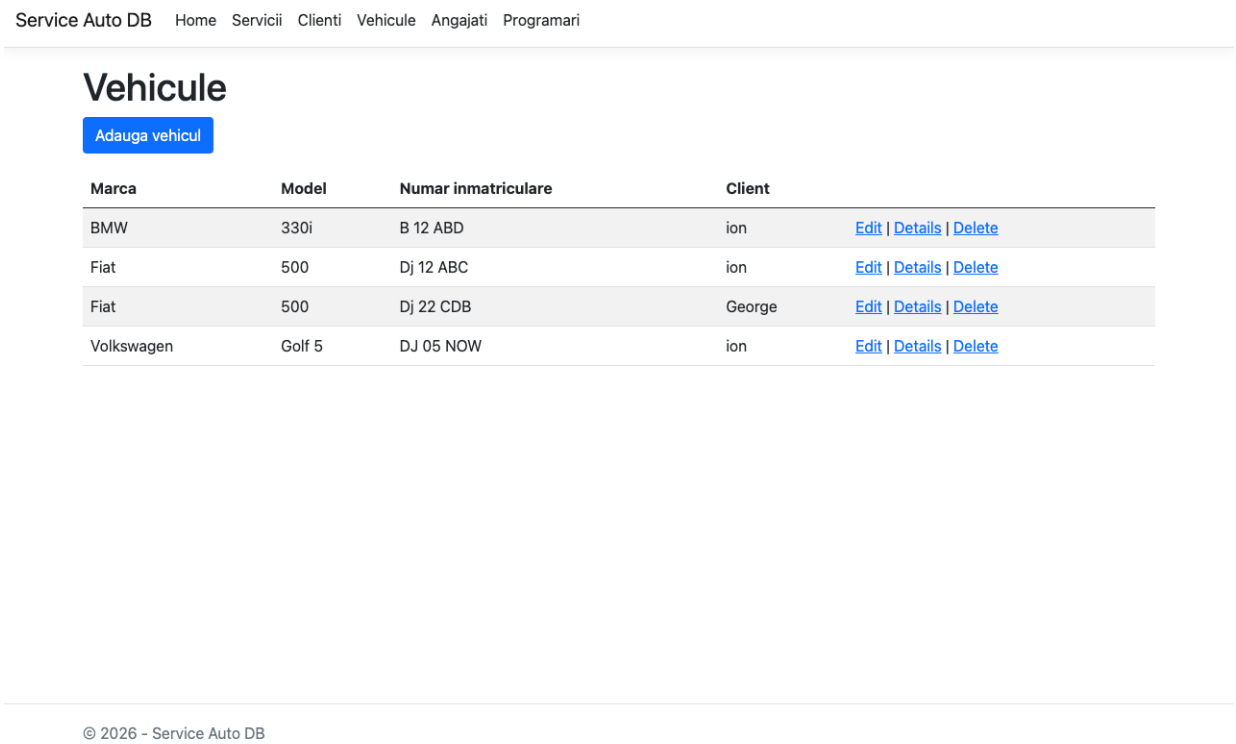


Figura 19. Listarea vehiculelor.

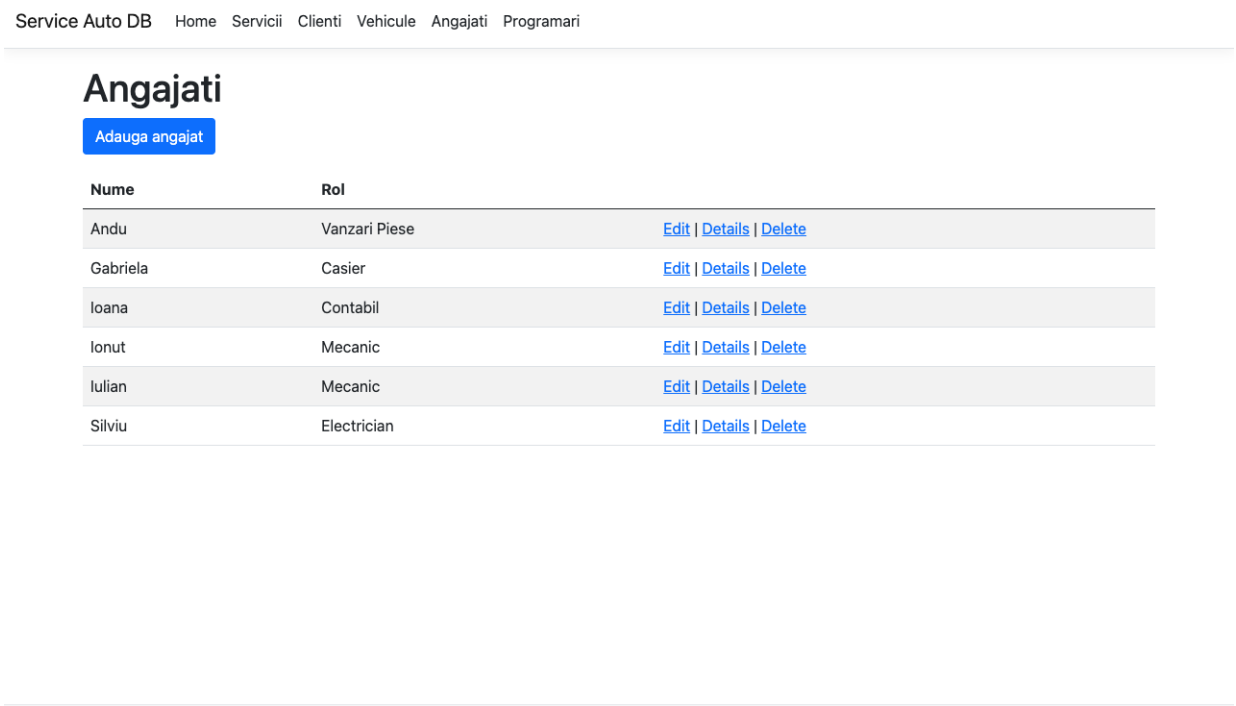


Figura 20. Administrarea angajatilor.

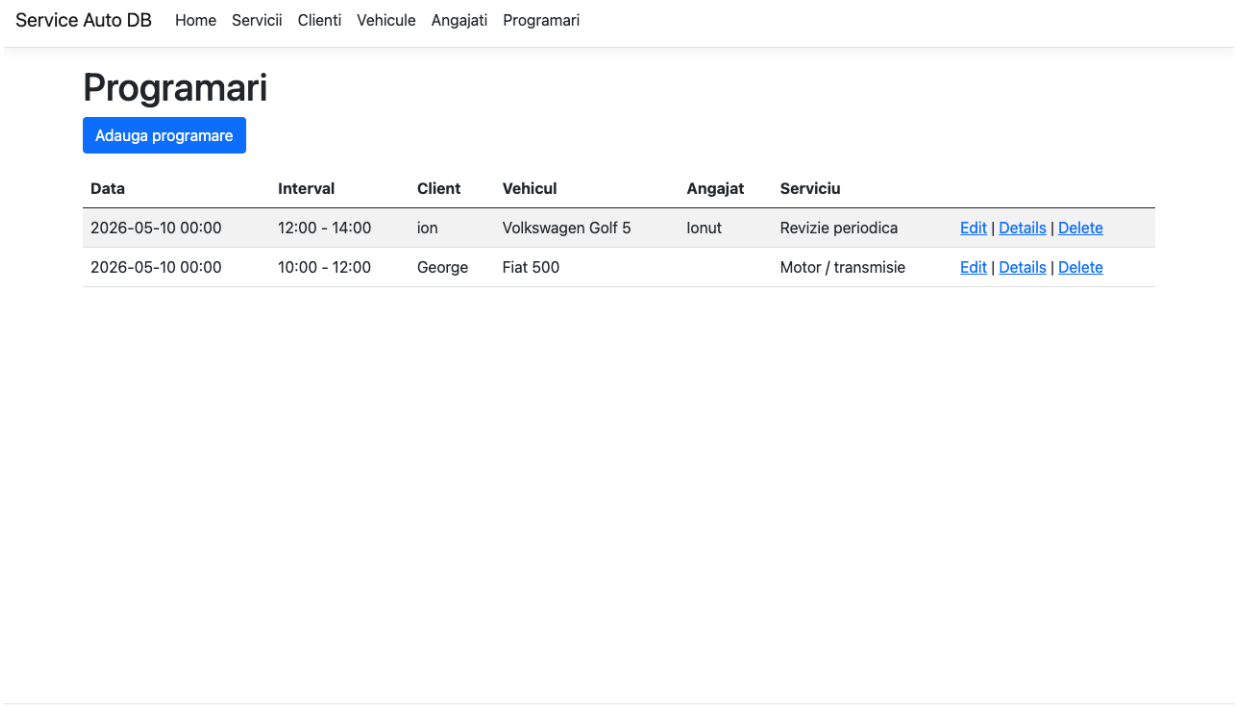


Figura 21. Administrarea programarilor.

Componenta API	Rol
AuthApiController	Inregistrare si autentificare client.
ProfileApiController	Vehicule si programari asociate clientului.
AppointmentsApiController	Crearea programarilor din site-ul public.

5. Ce metode de testare ati folosit pentru aplicatia voastra? Descrieti procesul de testare, precum si o problema identificata si solutia aplicata pentru rezolvarea sa.

Metode de testare

Metoda	Ce am verificat	Unde s-a aplicat
Testare manuala functionala	Scenariul complet: inregistrare, login, profil, vehicul, programare.	Frontend + API
Testare CRUD	Create, Read, Update, Delete pentru entitatile din panoul MVC.	Views si controllere MVC
Testare API	Coduri HTTP, payload JSON si raspunsuri de eroare.	Endpoints /api/*, curl/Postman
Testare integrare	CORS, mesaje de eroare, date afisate corect in browser.	appointments.js, profile.js
Testare validare	Campuri obligatorii, email invalid, lipsa vehicul.	Formulare si ModelState
Testare regresie	Pornirea aplicatiei pe o baza de date deja existenta.	Program.cs, migrari EF Core

Proces de testare - scenariul principal

1. Pornirea backend-ului cu dotnet run --urls http://localhost:5099.
2. Pornirea frontend-ului static cu python3 -m http.server 8000 din folderul frontend/.
3. Crearea unui cont nou in register.html.
4. Autentificarea contului si verificarea schimbarii meniului.
5. Adaugarea unui vehicul in profile.html si verificarea aparitiei lui in lista.
6. Deschiderea appointments.html, completarea datei, intervalului, serviciului si vehiculului.
7. Trimiterea formularului si verificarea mesajului de succes cu ID-ul programarii.
8. Verificarea in /Appointments ca programarea apare si in panoul MVC.
9. Anularea programarii din profil si verificarea disparitiei din lista.

Problema identificata si solutie

In timpul integrarii formularului public de programari cu o baza de date creata mai devreme, inserarea esua deoarece tabelul Appointments nu avea inca noua coloana TimeSlot. Practic, codul aplicatiei fusese actualizat, dar unele baze locale ramasesera in schema veche.

Solutia a fost sa adaug un mecanism de upgrade incremental: AppointmentSchemaUpgrader.EnsureTimeSlotColumnAsync(), apelat la pornire in Program.cs. Acesta

verifica structura tabelului prin PRAGMA table_info("Appointments") si adauga TimeSlot doar daca lipseste. O abordare asemanatoare a fost folosita pentru PasswordHash pe Customers.

```
PRAGMA table_info('Appointments');
ALTER TABLE Appointments ADD COLUMN TimeSlot TEXT;
```

Dupa repornirea aplicatiei, fluxul de programare a functionat pe baza de date existenta, fara stergerea manuala a fisierului serviceauto.db si fara pierderea datelor locale.

6. Elaborati un plan de promovare a aplicatiei voastre.

Promovarea aplicatiei trebuie legata de comportamentul real al clientilor unui service: multi ajung prin recomandari locale, Google Maps/Business Profile sau revin periodic pentru revizii. De aceea, planul porneste de la actiuni mici, usor de aplicat, care pot creste treptat numarul de programari online.

Public tinta

- Proprietari auto din zona locala a service-ului.
- Clienti recurenti care fac revizii periodice.
- Soferi care prefera sa trimita o cerere online in locul apelului telefonic.
- Firme mici cu flote auto care au nevoie de programari predictibile.

Canale si actiuni

Faza	Actiune	Detalii
Lansare	QR code in service	Afis la receptie si pe facturi cu link direct catre programari.
SEO local	Google Business Profile	Actualizare program, servicii, link catre site si fotografii reale.
Social media	Facebook / Instagram	Postari cu servicii, preturi orientative, sfaturi de intretinere si imagini before/after.
Offline	Flyere si bonuri	Mesaj simplu: „Programeaza-te online in 3 pasi”.
Fidelizare	Reminder email/SMS	Extensie viitoare: notificare cu 24h inainte de programare.
Parteneriate	Flote / firme locale	Pachet de revizii si programari centralizate pentru companii.

Indicatori de succes

- Numar de programari online pe luna.
- Rata de conversie vizitator → cont creat.
- Reducerea apelurilor telefonice pentru programari simple.
- Timpul mediu de completare a formularului.
- Numarul de clienti care revin pentru a doua programare.

Buget orientativ

Element	Estimare
Materiale locale: afise, QR, flyere	cost redus / ocazional

Element	Estimare
Campanie social media locala	200-500 RON/luna, optional
Domeniu .ro + hosting	aprox. 150-300 RON/an
Timp administrare continut	1-2 ore/saptamana

7. Descrieti aspectele de utilizabilitate si accesibilitate relevante pentru aplicatia voastra; cum ati putea sa le integrati / implementati?

Utilizabilitate

Aspect	Stare in proiect	Imbunatatire propusa
Navigare clara	Navbar constant pe pagini si CTA vizibil pe homepage.	Breadcrumbs in Profil si Programari.
Feedback la actiuni	Alerte success/danger dupa raspunsurile API.	Toast-uri temporare si mesaje langa campurile cu erori.
Flux progresiv	Clientul trece prin login → vehicul → programare.	Wizard in 3 pasi cu indicator de progres.
Consistenta vizuala	Paleta verde cu accent galben, Bootstrap 5.	Mic design system pentru butoane, carduri si formulare.
Mobile-first	Navbar responsive si layout pe grid.	Testare pe telefon real si touch targets de minimum 44px.
Prevenire erori	Formular dezactivat fara autentificare sau fara vehicul.	Validare inline inainte de submit.

Accesibilitate

Din punct de vedere al accesibilitatii, aplicatia are nevoie de atentie in special pe formulare, mesaje de feedback si navigare cu tastatura. Pentru un site de programari, utilizatorul trebuie sa poata intelege rapid ce camp a gresit si ce pas urmeaza.

- Etichete label corelate corect cu input-urile prin atributul for/id.
- Mesaje de eroare anuntate cu role="alert" sau aria-live="polite".
- Contrast suficient intre text, fundal si butoane, mai ales pentru verde si galben.
- Focus vizibil pentru linkuri si butoane, pentru navigare cu tastatura.
- Texte alternative pentru imaginile importante si capturi folosite in documentatie.
- Ordine logica a titlurilor H1, H2, H3 pe fiecare pagina.

8. Descrieti aspectele de securitate relevante pentru aplicatia voastra; cum ati putea sa le integrati / implementati?

Securitatea aplicatiei este importanta deoarece sistemul lucreaza cu date personale, masini si programari. Varianta actuala include cateva masuri corecte pentru laborator, dar are si zone care trebuie intarite inainte de productie.

Aspect	Situatie actuala	Recomandare
Parole	Hash cu IPasswordHasher<Customer>.	Pastrare abordare + politica de parola mai stricta.
Autentificare API profil	Clientul este identificat prin email in query/body.	JWT sau cookie authentication; email extras din token, nu primit de la client.
Panou MVC	Fara autentificare de administrator.	ASP.NET Core Identity + rol Admin + [Authorize].
HTTPS	UseHttpsRedirection activ.	Certificat valid pe serverul de productie.
CORS	Configurare permisiva pentru development.	Limitare la domeniul frontend in productie.
CSRF	MVC poate folosi antiforgery token.	Pentru API: token sau SameSite cookies.
Validare input	Data Annotations si ModelState.	Limitare lungime note si validari suplimentare server-side.
SQL Injection	EF Core parametrizat.	Evitare SQL raw neparametrizat.
XSS	Razor encodeaza implicit.	Evitarea innerHTML sau escapare explicita in frontend.

Plan de hardening pentru productie

- Adaugarea autentificarii JWT sau cookie-based dupa login.
- Inlocuirea identificarii prin email trimis din client cu identificarea din token.
- Protejarea panoului MVC cu roluri Client/Admin.
- Rate limiting pe /api/auth/login impotriva atacurilor brute-force.
- Logging audit pentru operatii CRUD sensibile.
- Backup periodic pentru serviceauto.db sau migrare la SQL Server/PostgreSQL managed.

9. Enumerati limitarile si punctele slabe ale aplicatiei voastre, precum si imbunatatirile / extinderile care s-ar putea realiza ulterior.

Limitari actuale

- Panoul MVC nu are autentificare, deci intr-un scenariu public ar permite modificari neautorizate.
- Sesiunea clientului este pastrata in localStorage, ceea ce nu ofera control server-side sau expirare reala.
- API-ul de profil identifica utilizatorul prin email transmis din client, nu printr-un token sigur.
- Nu exista teste automate, deci regresiile pot aparea dupa modificari mai mari.
- Programarile nu verifica inca disponibilitatea reala a intervalelor orare.
- Interfata publica permite alegerea unui singur serviciu, desi modelul bazei de date suporta mai multe servicii per programare.
- Frontend-ul static si backend-ul ruleaza pe porturi diferite in development, deci depind de CORS si configurare manuala.
- SQLite este potrivit pentru demo si laborator, dar poate deveni limitat la multe scrieri concurente.

Extinderi propuse

Prioritate	Functionalitate propusa	Impact
Inalta	Autentificare JWT + roluri Client/Admin.	Creste securitatea si separa clar zonele aplicatiei.
Inalta	Calendar de disponibilitate si validare anti-suprapunere.	Reduce programarile conflictuale.
Medie	Notificari email/SMS de confirmare si reminder.	Scade rata de neprezentare.
Medie	Istoric lucrari si facturare per vehicul.	Transforma aplicatia intr-un instrument complet pentru service.
Medie	Frontend mai avansat, de exemplu React sau Razor unificat.	Reduce logica duplicata si imbunatateste experienta utilizatorului.
Scazuta	Rapoarte: programari/luna, servicii populare, venituri estimate.	Ajuta deciziile de business.
Scazuta	Versiune in limba engleza.	Utila pentru extindere sau prezentare internationala.

10. Autoevaluare (obligatoriu, fara punctaj)

Intrebare	Raspuns de completat
Nota pe care consider ca o merit la laborator	9
Nota pe care consider ca o merit la examen (documentul curent)	6
Nota finala pe care consider ca o merit la disciplina PAW	7
Numar de ore alocat pentru elaborarea documentului	5
Resurse folosite pentru elaborarea documentului	Codul proiectului propriu, cerinta de examen, materialele de laborator/curs, documentatie tehnica pentru ASP.NET Core, EF Core si Bootstrap.
Prefer acest tip de evaluare sau un examen clasic?	Prefer acest tip de evaluare, examenul propriu-zis necesita mai mult timp pentru pregatire si consider ca acest tip de evaluare este mai constructiv decat un examen scris pe foaie.
Impresii generale despre laborator & curs si sugestii	Prezentarile si structurarea laboratorului sunt foarte bine puse la punct.

Anexa - Structura repository-ului

Structura proiectului este separata intre frontend-ul static, aplicatia ASP.NET Core si documentatia proiectului:

```
service-auto/
├── frontend/          # Site static pentru client
├── ServiceAutoApp/    # Backend MVC + API
│   ├── Controllers/
│   ├── Services/
│   ├── Repositories/
│   ├── Models/
│   ├── Data/
│   ├── Migrations/
│   └── Views/
└── docs/
    ├── PROIECT-EXAMEN-PAW-2025.md
    └── screenshots/
```